The `liftcomplex` obstruction: why auxiliary–variable lifting fails for the bubble at imaginary μ

Tropical Monte Carlo (FINAL4) — planAU Phase-0 finding

2026-06-29

Abstract

The auxiliary–variable *lift* (v3 engine, `ProcessSectorLifted`) folds a large kinematic coefficient into the Newton polytope so the tropical sampling map matches a hierarchical integrand. It is the intended fix for the noisy imaginary–μ "principal series" (FLAG B). On the actual bubble integrand at imaginary μ the lift refuses with `TropicalEval::liftcomplex` on every cone. This note explains *exactly why*: the lift introduces a real *domain inequality* that the ordinary (unlifted) algorithm does not have, and that inequality is governed by an effective exponent $\tilde{a}$ built from *both* the polynomial exponents B_j *and* the monomial exponents m_i. The `SplitRealImag` complex–exponent mode removes the imaginary part of B_j before the inequality is formed, but *not* that of m_i; the bubble at imaginary μ is the first integrand with complex *monomial* exponents, so $\tilde{a}$ stays complex and the lift aborts. The flattening (change of variables) is *not* the broken step. A one–character experiment confirms the diagnosis and points to a localized fix.

1 Setup and notation

Each presector integrand has the form

$$I \;=\; \int_{[0,\infty)^n} \prod_{i=1}^{n} x_i^{m_i} \; \prod_j P_j(x)^{\,B_j} \; \mathrm{d}^n x, \tag{1}$$

with *monomial exponents* m_i (on the bare powers $x_i^{m_i}$) and *polynomial exponents* B_j (on the polynomials P_j). For the bubble these exponents are functions of the propagator dimensions $\delta_\pm = \frac{3}{2} \pm \mu$ and the regulator d_ϵ. For *real* μ they are real; for *imaginary* μ (the principal series) they are **complex**, and — this is the crux — *both* families m_i and B_j acquire nonzero imaginary parts.

Concrete instance. For `bubblepm` presector 1 at $\mu = 3i/2$,

$$\mathrm{Im}(m) \;=\; \big\{ -\tfrac{3}{2},\ \tfrac{3}{2},\ -\tfrac{3}{2},\ \tfrac{3}{2},\ 3 \big\}, \qquad \mathrm{Im}(B) \neq 0 \text{ as well.}$$

2 The ordinary algorithm never *decides* on an exponent value

The tropical decomposition triangulates the dual fan into simplicial cones. In each cone it applies a *monomial change of variables*

$$x_i \;=\; \prod_a y_a^{(M)_{ia}}, \tag{2}$$

where the integer matrix M is fixed by the cone's rays — i.e. by the *support* (the set of exponent vectors), *not* by the exponent *values*. Substituting (2) into (1) and clearing the leading monomial from each polynomial is the *flattening* step (`ProcessSector`). The exponents simply ride along as numbers; the integrand is evaluated as $\exp\big(B_j \log P_j\big)$, which is well defined for *any* complex B_j.

> *Nothing in the unlifted pipeline asks a yes/no question about an exponent. Complex exponents flow through untouched — which is precisely why the unlifted scan of the imaginary–μ masses completed normally.*

3 The lift adds a real *domain inequality*

To tame a hierarchical coefficient $C\,x^\alpha$ the lift rewrites it as $c\,z^k x^\alpha$, promotes the anchor z to a genuine integration variable, and pins it with a delta function:

$$C\,x^\alpha \;\longrightarrow\; c\,z^k x^\alpha, \qquad I \;=\; \int_{[0,\infty)^{\,n+1}} (\cdots)\;\delta(z - z_0)\,\mathrm{d}z\,\mathrm{d}^n x, \qquad z_0 = |C|^{1/k}. \tag{3}$$

One flattens this $(n{+}1)$–dimensional problem and then **resolves the delta**: a pivot variable y_p is chosen and eliminated, folding the integral back to n dimensions. That fold–back is the new ingredient. Solving $\delta(z - z_0)$ for the pivot turns the sector's domain into a *cut region*, emitted as an indicator

$$\mathrm{Boole}\big[\,\log y_p^\star \le 0\,\big], \qquad \log y_p^\star \;=\; \frac{\log z_0 - \sum_a \mathrm{ic}_a\,\log y_a}{m_p}, \tag{4}$$

together with a classification of the sector (`HasConstantTerm`, `EmptyDomain`, divergent/convergent). The coefficients in (4) and the classification are read off the **effective exponents** $\tilde{a}$ left after the delta is resolved.

Why $\tilde{a}$ must be real. Equation (4) is an *inequality*: it carves $[0,1]^d$ into a region that contributes and a region that does not. "Is $(2 - 3i) \le 0$?" is meaningless. Hence the lift requires $\tilde{a} \in \mathbb{R}$. The engine guards this explicitly; when every admissible pivot yields a complex $\tilde{a}$ it aborts:

```
TropicalEval::liftcomplex = "ProcessSectorLifted: cone ‘1‘ -- all candidate
  pivots produce complex atilde; cannot emit a real-valued domain indicator."
```

Crucially, $\tilde{a}$ depends on *both* exponent families. Under the change of variables (2) the bare factor $\prod_i x_i^{m_i}$ becomes

$$\prod_i x_i^{m_i} \;=\; \prod_a y_a^{\sum_i m_i (M)_{ia}}, \tag{5}$$

so the new variables y_a inherit exponents $\sum_i m_i (M)_{ia}$ that are **linear in the monomial exponents** m_i. These combine with the polynomial–derived exponents; the delta resolution then produces

$$\tilde{a} \;=\; \tilde{a}\big(m,\, B,\, \text{cone geometry}\big), \qquad \text{linear in } m \text{ and } B. \tag{6}$$

If *any* contributing exponent is complex, $\tilde{a}$ is complex and the inequality (4) is ill posed.

4 `SplitRealImag`: the intended fix, wired to only half the inputs

The engine's mechanism for complex exponents is `ComplexExponentMode -> "SplitRealImag"`: build the (real) domain map from the *real parts* of the exponents, and peel the *imaginary parts* off into a pure oscillatory phase

$$\exp\Big(i \sum_j \mathrm{Im}(B_j) \left(\log Q_j + \mathrm{MonoFactorLog}_j \right) \Big) \tag{7}$$

that multiplies the integrand. A phase does not move the boundary, so (4) stays real. This is correct and is validated for complex *polynomial* exponents (`cc_21b`, `cc_45`).

The defect is that the split is applied to only one of the two inputs to (6). In `EvaluateTropicalMC` (file `tropical_eval.wl`, $\approx$ lines 3772 and 3814):

```
imBList  = Im[integrandSpec["PolynomialExponents"]];      (* polynomial only *)
hasImagB = AnyTrue[imBList, (! TrueQ[PossibleZeroQ[#]]) &];
...
ProcessSectorLifted[
  MapAt[Re, liftedSpec, {Key["PolynomialExponents"]}],     (* Re of B only;  *)
  ...                                                       (* m left complex *)
  "Eps" -> eps]
```

Only $\mathrm{Re}(B_j)$ is taken; the monomial exponents m_i are passed through **complex**. By (5)–(6), $\tilde{a}$ is therefore *still* complex — now from the monomial half — and the `liftcomplex` guard fires on every cone.

> *The bubble at imaginary μ is the first integrand anyone has lifted whose* monomial *exponents are complex; all prior complex tests had complex B_j with real m_i, so the incomplete split was invisible upstream.*

5 Empirical confirmation

On the `bubblepm` $\mu = 3i/2$ lifted spec (presector 1; n=5, so 611 six–dimensional cones), processing each cone two ways:

What is fed to `ProcessSectorLifted`	complex $\tilde{a}$?	Result
Re of *polynomial* exponents only *(what the engine does)*	yes	`liftcomplex` — **every cone fails**
Re of *both* monomial *and* polynomial exponents	no	**all 611 cones succeed**

So the obstruction is *solely* the un–split monomial exponents, and making $\tilde{a}$ real is sufficient to let the decomposition through. (For reference, the unlifted path and an independent `NIntegrate` of (1) both evaluate this point without issue, giving $\approx 1.27 \times 10^{-14} + 1.40 \times 10^{-15}\, i$.)

6 What is and is not broken

- **Flattening / change of variables — NOT broken.** It is geometric (fixed by the cone rays), shared verbatim with the unlifted path, and handles complex exponents fine. The imaginary parts *originate* in the flattened exponents via (5), but storing complex numbers is harmless.

- **Delta–resolution / pivot / domain indicator (lift–only) — broken.** This step needs $\tilde{a} \in \mathbb{R}$ to form the inequality (4) and to classify the sector; with complex m_i it refuses (`liftcomplex`).

7 The fix (sketch)

Extend `SplitRealImag` to treat the monomial exponents on the same footing as the polynomial exponents:

1. **Decomposition side.** Take $\mathrm{Re}(m_i)$ as well as $\mathrm{Re}(B_j)$ before `ProcessSectorLifted`, so $\tilde{a}$ in (6) is real and the domain indicator is well posed. *(Shown sufficient above:* 611/611 *cones.)*

2. **Phase side.** Carry $\mathrm{Im}(m_i)$ into the *same* oscillatory phase that already transports $\mathrm{Im}(B_j)$. Using (2),
$$\sum_i \mathrm{Im}(m_i)\, \log x_i \;=\; \sum_a \Big(\textstyle\sum_i \mathrm{Im}(m_i)\, (M)_{ia} \Big) \log y_a,$$
i.e. the monomial imaginary parts contribute $\sum_i \mathrm{Im}(m_i)(M)_{ia}$ to the phase coefficient of $\log y_a$ in every sector (plus the matching anchor/MonoFactorLog constant from the lift). This is the part requiring care: it must be *numerically correct*, not merely non–crashing, and must reduce to the current behaviour when $\mathrm{Im}(m_i) = 0$ (so real–μ and prior complex–B outputs stay byte–identical).

The change is local to the lifted `SplitRealImag` path and its codegen (both the convergent route, e.g. `bubblepm`, and the divergent IBP / subtraction route, e.g. `bubblepp`); flattening is untouched. Validation should mirror `cc_21b`: (a) lifted `SplitRealImag` reproduces `NIntegrate` of (1) to $<$ 1–2%; (b) dropping the new monomial phase term gives a *wrong* answer (so it is load–bearing); (c) real–μ results are unchanged.